

—PROJECT STAGE II—

HARDWARE ACCELERATION OF OBJECT DETECTION USING PYNQ-Z2 BOARD

By

Kanchan Kumar Kaity (10200320005)

Anjisnu Chanda (10200320007)

Dhiraj Chandra (10200320019)

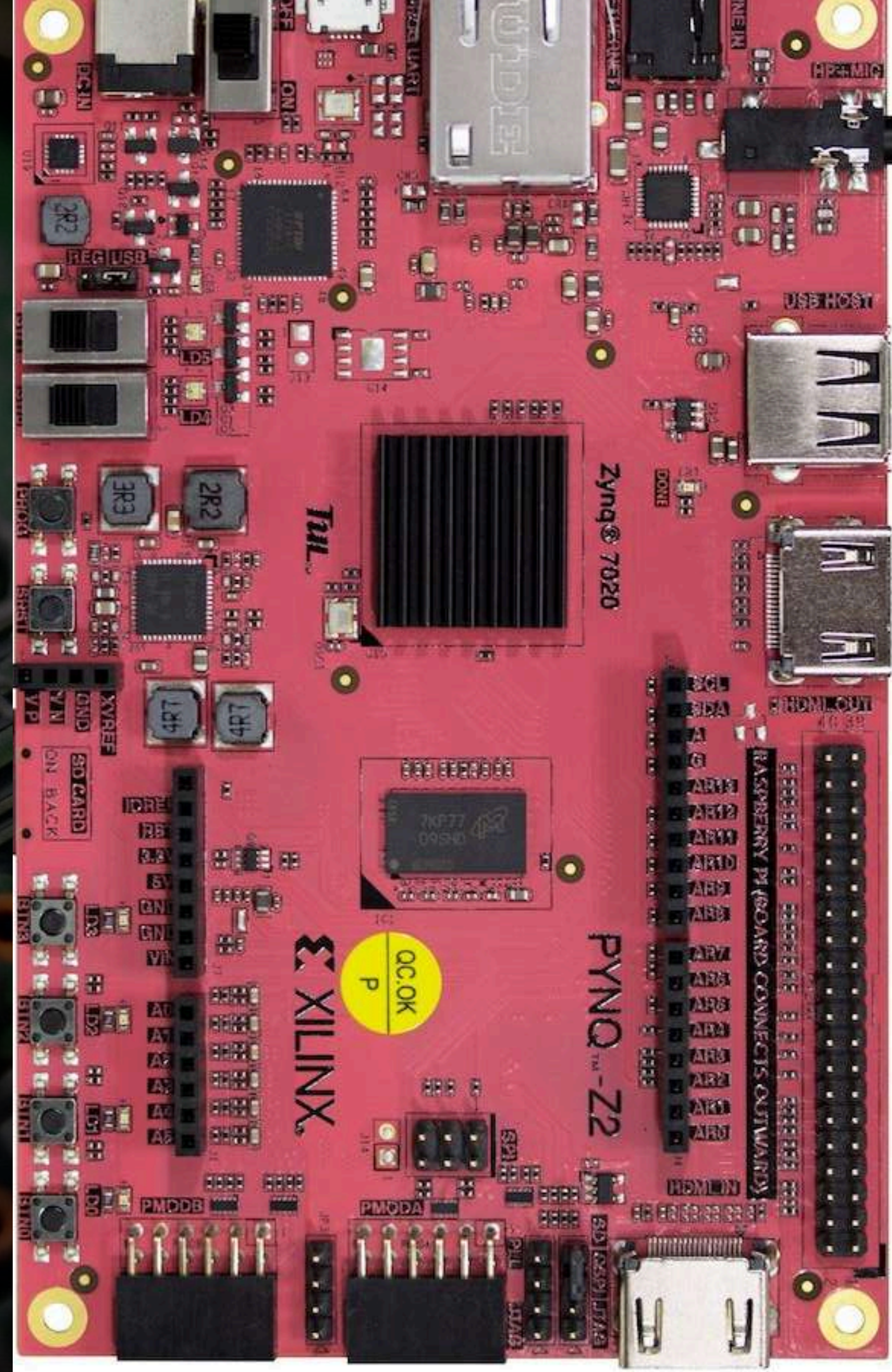
Debjoyoti Sardar (10200320020)

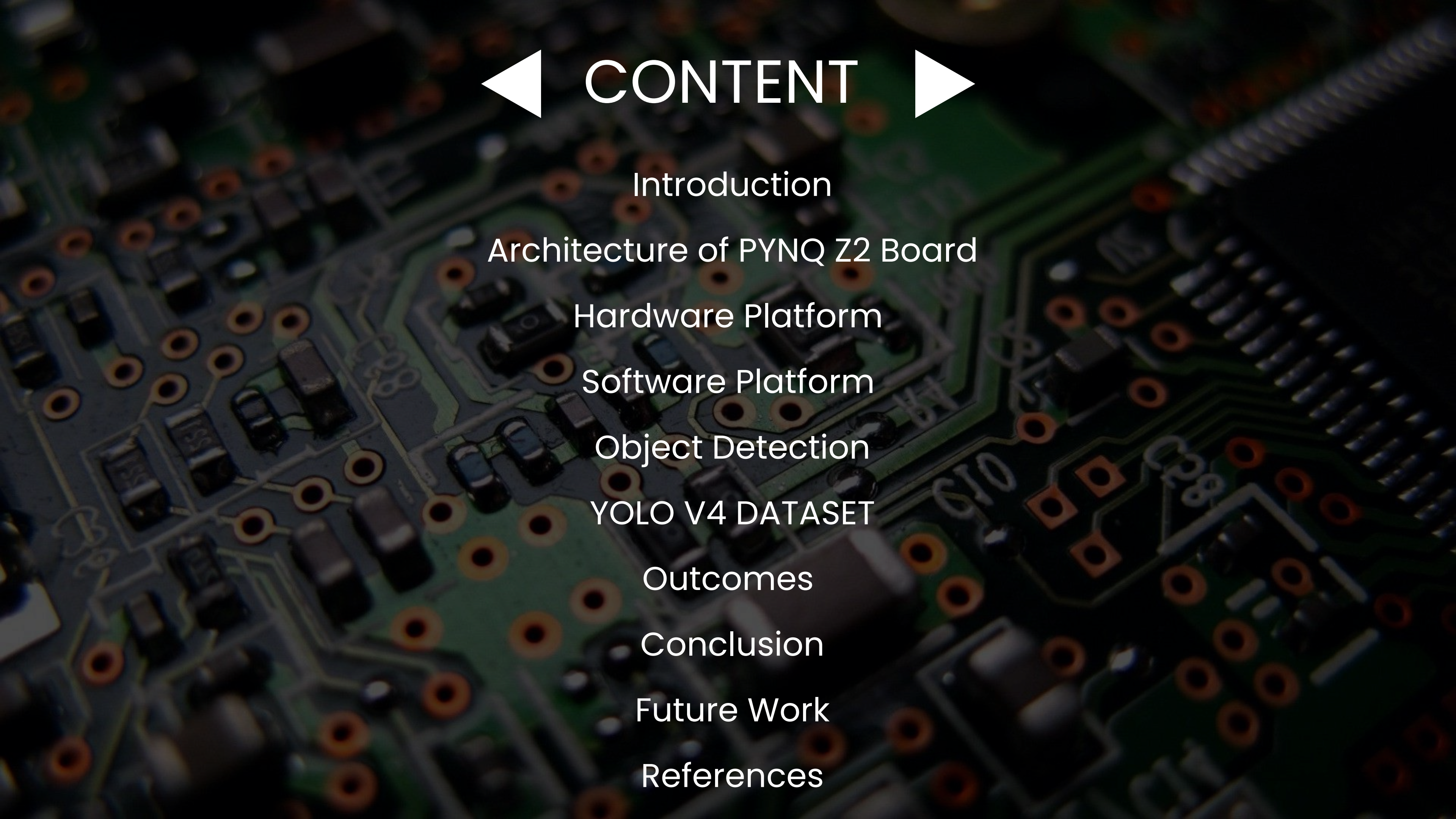
Supervised by : Dr. Himadri Sekhar Dutta

Date: 29/05/2024

TOPIC :

HARDWARE ACCELERATION OF OBJECT DETECTION USING PYNQ-Z2 BOARD





◀ CONTENT ▶

Introduction

Architecture of PYNQ Z2 Board

Hardware Platform

Software Platform

Object Detection

YOLO V4 DATASET

Outcomes

Conclusion

Future Work

References

INTRODUCTION

- This project aims to implement accelerated image processing algorithms on the PYNQ-Z2 board, leveraging its FPGA and ARM Cortex-A9 processor for real-time solutions in computer vision and medical imaging domains.
- This project in the multidisciplinary field of object detection focuses on utilizing fundamental and advanced techniques. Object detection using machine learning involves training a model to identify and classify objects within images or videos.
- The PYNQ-Z2 board, equipped with dual-core ARM processors and FPGA, designed for Xilinx Zynq-7000 SoC, offers versatility for diverse applications, facilitated by the PYNQ framework's simplicity with Python, Jupyter notebooks, and overlays—serving as a potent tool for education and prototyping at the FPGA-software development intersection.

Architecture of PYNQ Z2 Board

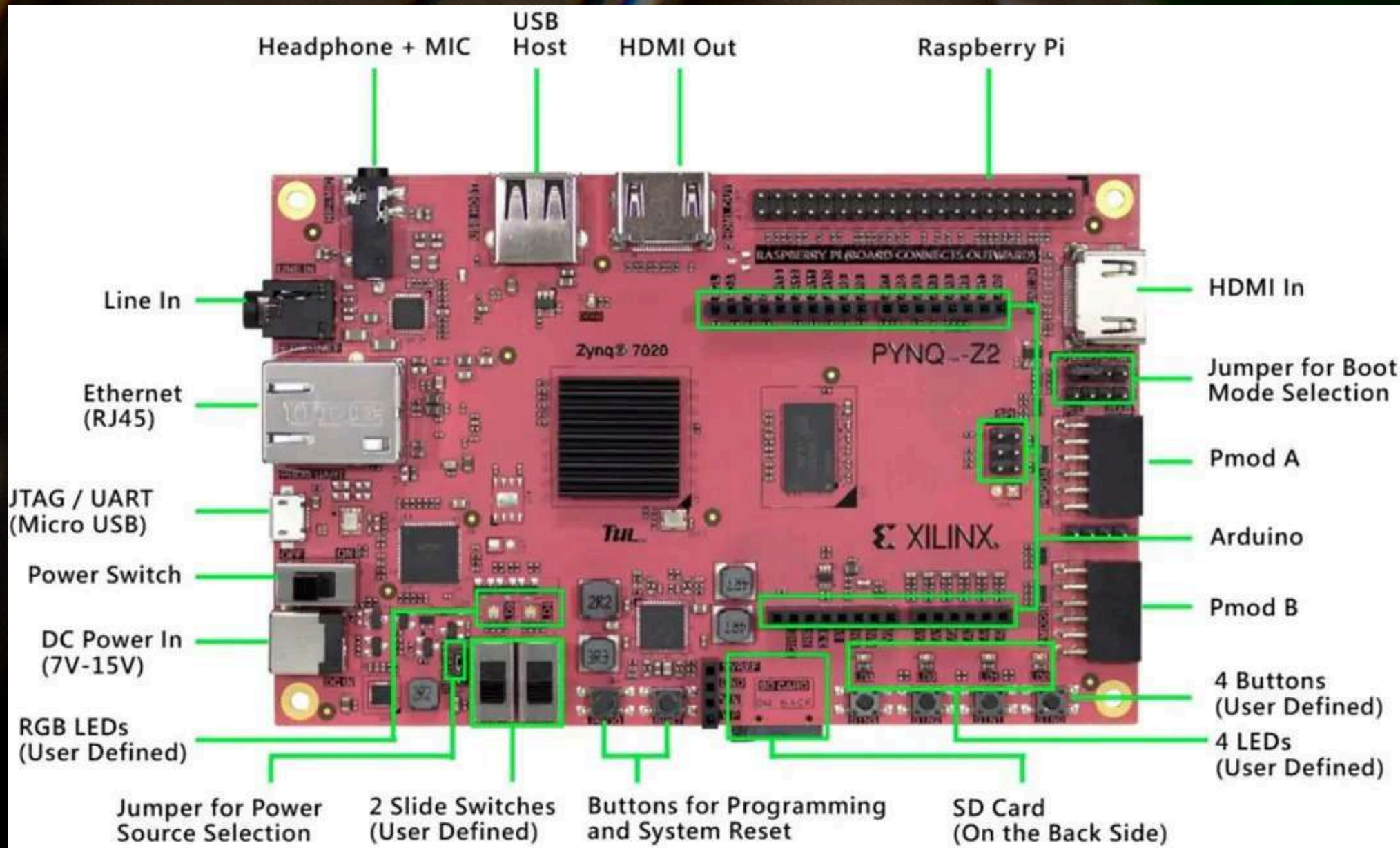


Fig 1: Architecture of Python Productivity for Zynq Board

1. Zynq-7000 SoC:

- Dual-core ARM Cortex-A9 processor
- Xilinx 7-series FPGA fabric

2. Memory:

- 512 MB DDR3 memory

3. Storage:

- MicroSD card slot for additional storage

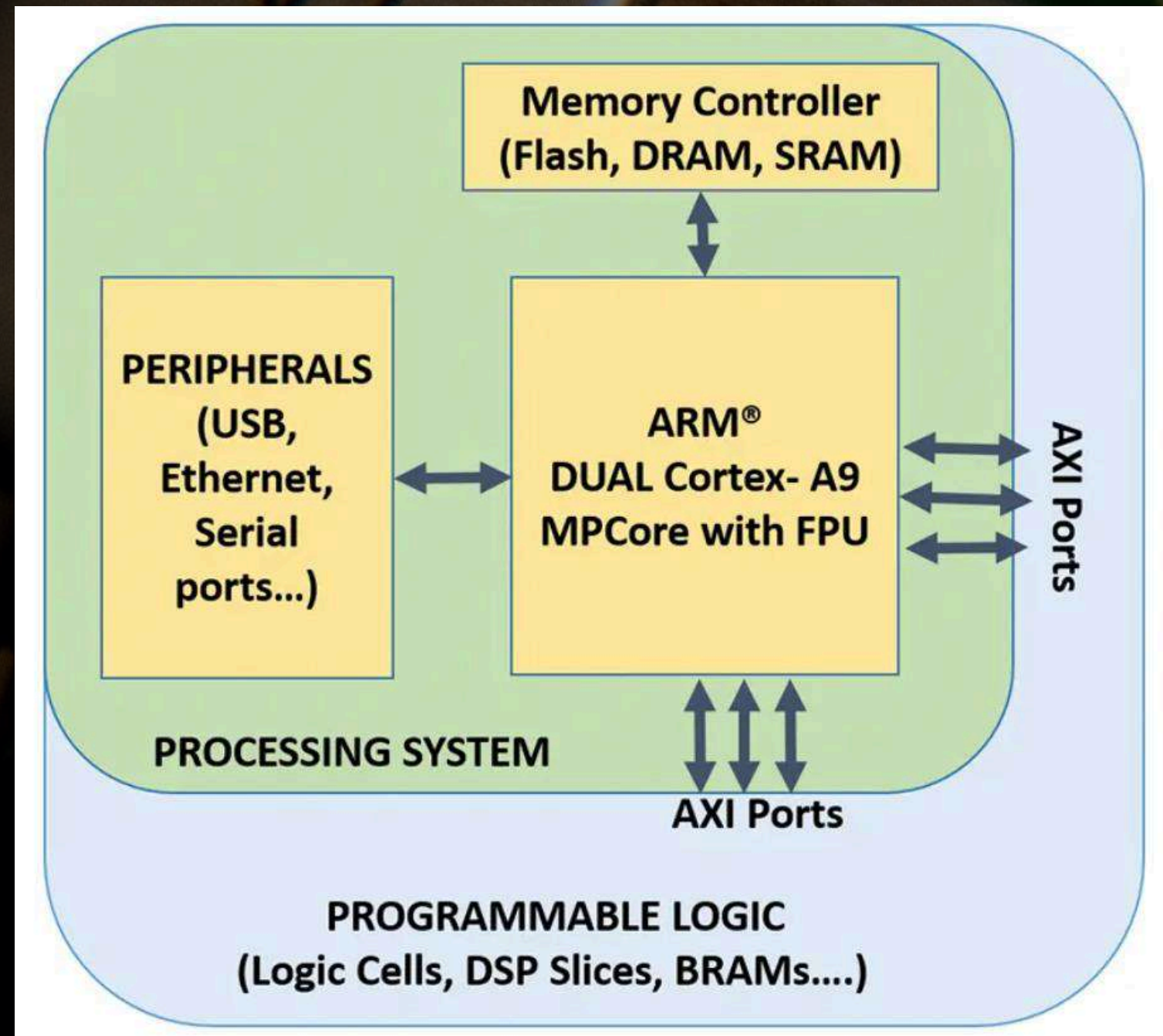
4. Connectivity:

- Gigabit Ethernet port
- USB 2.0 ports (Micro USB and USB Type-A)
- HDMI input and output ports
- Audio input and output jacks
- PMOD connectors for additional peripherals

5. Expansion Interfaces:

- Arduino and Raspberry Pi headers for additional hardware compatibility

Architecture of PYNQ Z2 Board



6. Programming and Configuration:

- Onboard JTAG programming and debugging interface
- Configuration over USB and JTAG

7. Power:

- Powered through a micro USB port or an external power supply

8. User Interfaces:

- Buttons (pushbuttons and slide switches) for user input
- RGB LEDs for visual feedback

9. Display:

- HDMI interface for connecting to external displays

10. Communication:

- WiFi and Bluetooth module for wireless communication (on some versions)

Fig 2 : Block Diagram of Python Productivity for Zynq Board

Hardware Platform

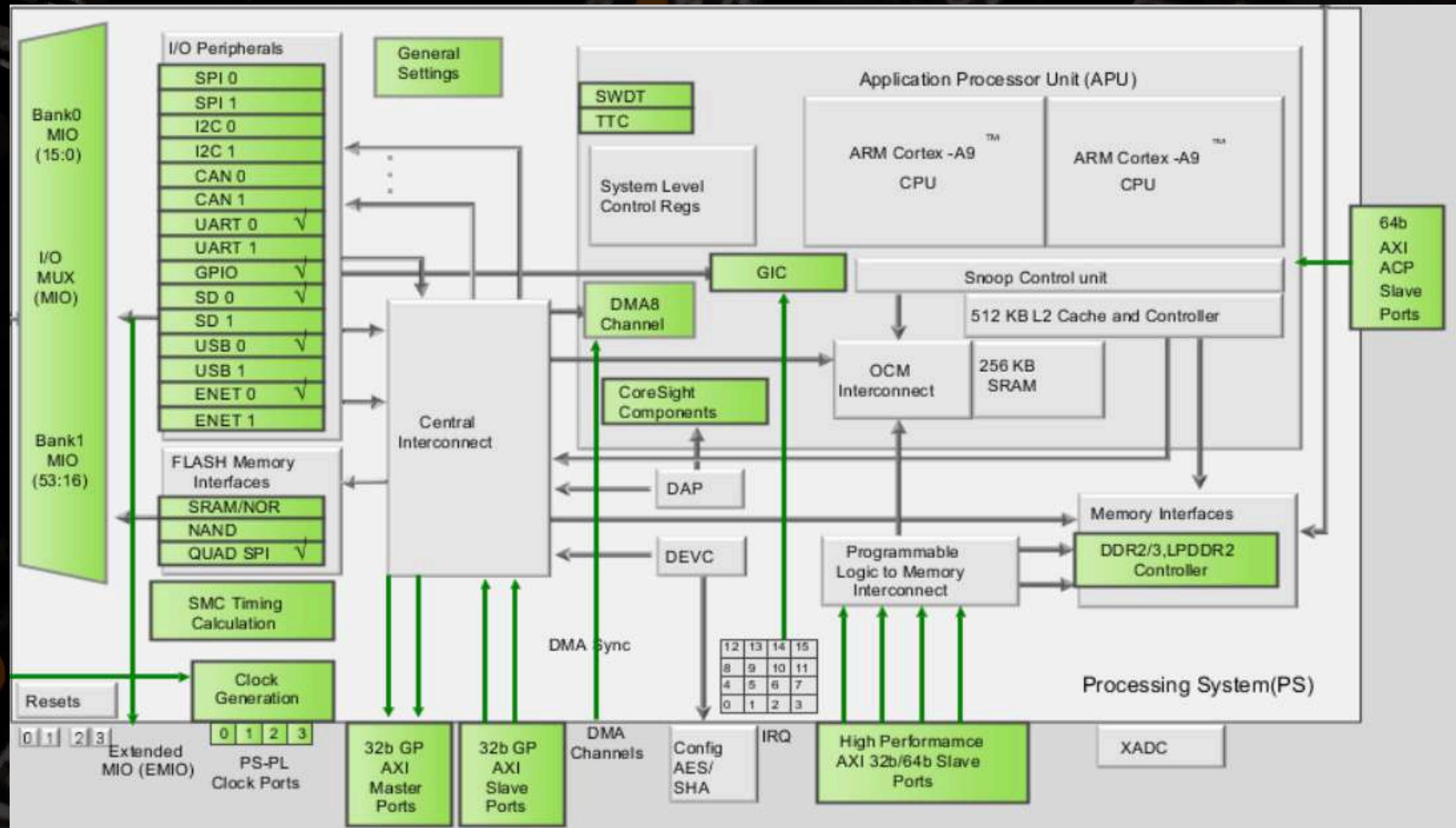


Fig 3: Overview of Hardware Platform:

Hardware Platform... contd

This device integrates Programmable logic block along with dual-core ARM Cortex-A9 Processing System

Key components include:

1. AXI4 Bus Architecture Configuration and connection of peripherals within programmable logic are done through this AXI bus architecture. It is mainly used to send data from ARM processing system to Programmable logic very fast.
2. DMA Controller: This enables efficient data transfer between programmable logic and ARM Cortex-A9 processor's memory. Implementing a DMA controller involves integrating the AXI DMA IP core.

Hardware Platform...contd

3. VDMA: The Video Direct Memory Access (VDMA) IP core is employed for video frame buffering and processing on the Pynq Z2, enhancing video-related applications.
4. AXI4 Interconnect: The AXI4 interconnect in Pynq Z2's programmable logic facilitates communication between different IP cores and the processing system, connecting peripherals, memory, and custom IP blocks.
5. Processor System Reset: Resetting the processor system and reconfiguring IP in the PYNQ Z2 involves a systematic shutdown, reprogramming IP, and reloading it with tailored parameters.
6. Concatenation IP: This strategic concatenation enhances parallel processing capabilities, creating a unified and powerful IP entity. It is also used for concatenation of various interrupt.

Hardware Platform...contd

7. Timing Controller (Video Timing Controller) : This IP block manages video timing signals, ensuring that video frames are synchronized correctly with the display device. It generates the necessary horizontal and vertical sync signals, as well as the pixel clock.
8. DVI to RGB Converter : Converts a DVI (Digital Visual Interface) signal to an RGB format. DVI is similar to HDMI but primarily used for video display purposes. This conversion is necessary to process or manipulate the video data in the RGB domain.
9. RGB to DVI Converter : Converts an RGB video signal to a DVI format. This conversion is necessary to output processed video data to a DVI-compatible display device.
10. Scaller IP : Adjusts the resolution of the video stream by scaling the input video to a different resolution. It can upscale or downscale the video to match the desired output resolution.

Software Platform

Xilinx Vivado and Jupyter Notebook have been used for this project.

- **Xilinx Vivado (2018.2)** : Xilinx Vivado is a comprehensive design suite for FPGA development, offering tools for synthesis, simulation, and analysis essential for configuring the PYNQ-Z2's programmable logic. It has been used to design the IP and overlay designs of the hardware and Programmable Logic (PL) blocks.
- **Jupyter Notebook** : It facilitates the development and testing of object detection algorithms, providing an easy-to-use interface to run Python code, manage data, and visualise results directly on the PYNQ-Z2 board. It has been used mainly to visualise the results and to operate and access the overlay design very efficiently without any prior knowledge of internal architecture.

OBJECT DETECTION

- Object detection, an intriguing facet of computer vision, finds its application in identifying and pinpointing objects within images or video streams. Utilizing the PYNQ-Z2, a robust development board housing a Xilinx Zynq-7000 system-on-chip (SoC), one can embark on captivating projects within this realm.
- To perform object detection on the PYNQ-Z2, load a pre-trained model onto the board. Capture images or video frames using the board's camera interface, such as HDMI input, USB camera, or an onboard camera module. Input these images into the deep learning model for inference, which will analyze and identify objects with bounding boxes.

OBJECT DETECTION BLOCK DIAGRAM

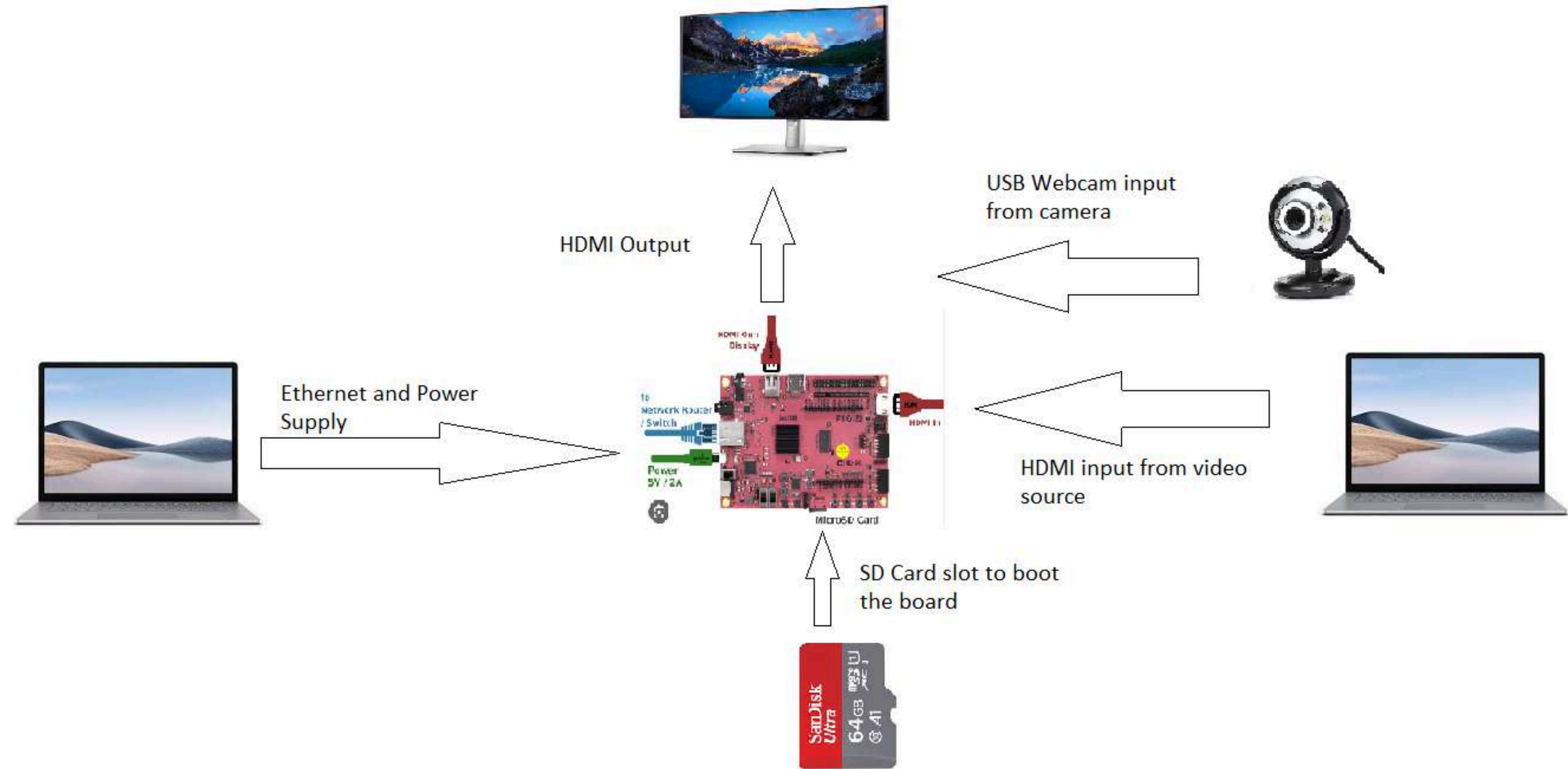


Fig 4: OBJECT DETECTION BLOCK DIAGRAM

PYNQ Z2 INTERNAL SYSTEM

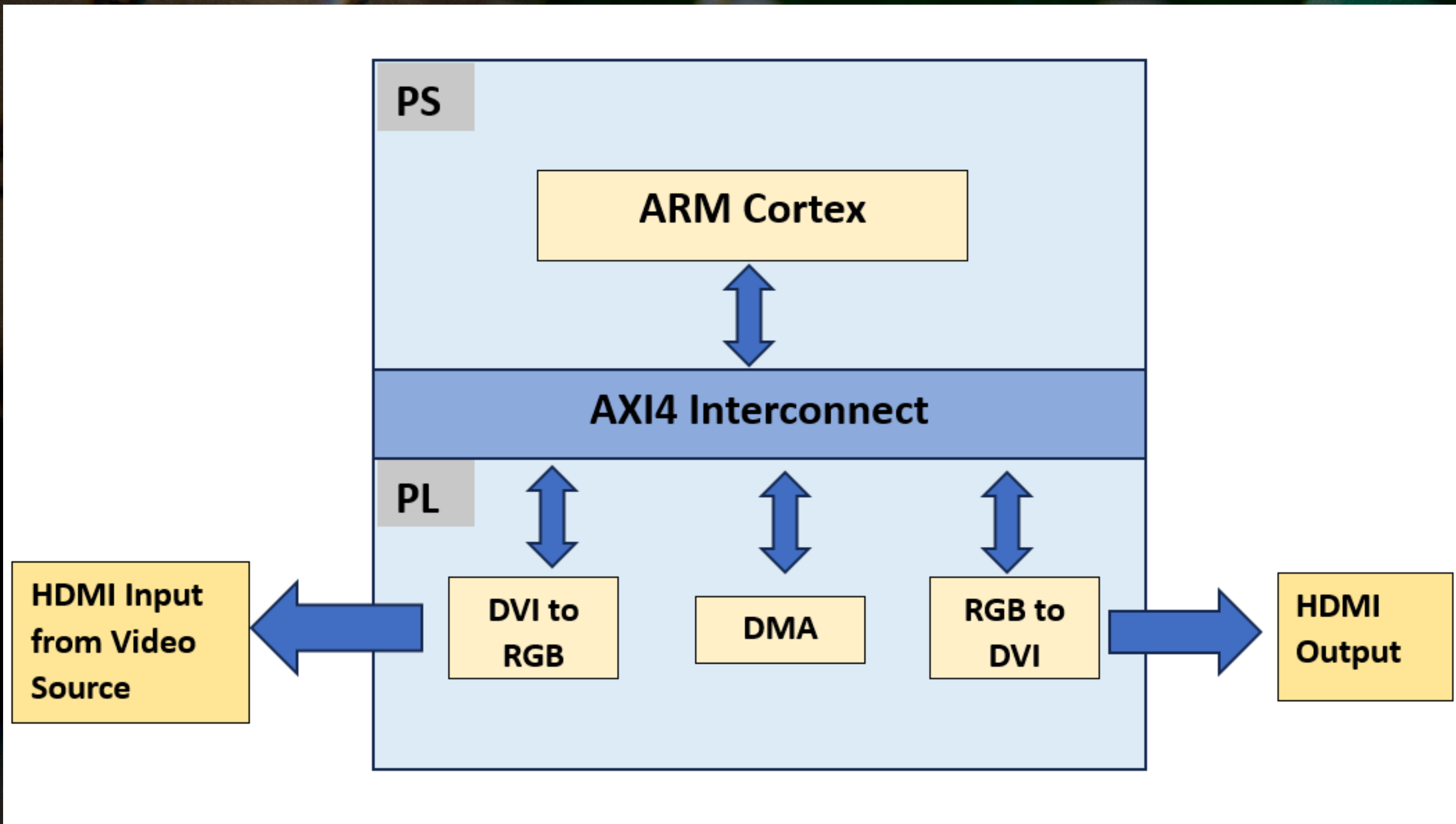


Fig 5: PYNQ Z2 Internal system

VIDEO STREAMING OVERLAY DESIGN

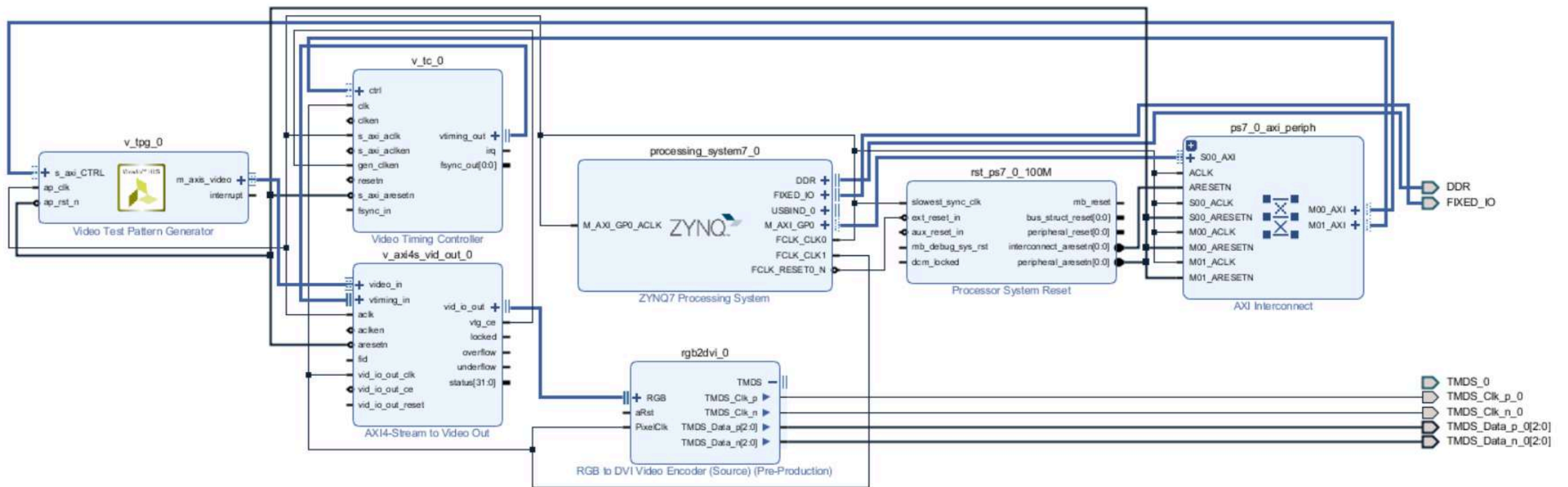


Fig 6: Overview of Video streaming overlay design (Xilinx Vivado)

YOLO V4 DATASET

YOLO (You Only Look Once)[5] is an object detection algorithm that's popular for its speed and accuracy. YOLOv4 is one of the latest versions, known for its improved performance compared to earlier versions.

Common datasets used for training object detection models like YOLOv4 include:

- **COCO (Common Objects in Context):**
- **Pascal VOC (Visual Object Classes):**
- **Open Images Dataset**
- **Custom Datasets**

To train a YOLOv4 model, we would typically use one of these datasets or a combination of them, depending on your specific requirements and the types of objects you need to detect.

SD CARD BOOTING

- To boot the PYNQ-Z2 board from an SD card, first, download the appropriate PYNQ-Z2 image from the PYNQ project website. Format an SD card (at least 8GB) using a tool like SD Card Formatter, then use Etcher or Win32 Disk Imager to write the image to the SD card.
- Insert the SD card into the board's slot, connect the board to your computer via USB for power and UART communication, and optionally connect an Ethernet cable for network features.
- Turn on the board and find the board's IP address if using Ethernet. Access the Jupyter Notebook interface by navigating to `http://<board_ip_address>:9090` in a web browser, using the default username and password Xilinx. Verify the installation by running example notebooks to ensure everything is working correctly.

ETHERNET CONNECTION

- To set up an Ethernet connection between a PYNQ-Z2 board and a laptop, start by connecting an Ethernet cable directly between the board and the laptop. Ensure the PYNQ-Z2 is powered on and connected to the laptop via a USB cable for serial communication.
- Insert the SD card into the PYNQ-Z2 board and power it on. On the laptop, configure the Ethernet adapter with a static IP address. For Windows, navigate to Network and Sharing Center, change adapter settings, and set the IP to 192.168.3.1 with a subnet mask of 255.255.255.0.
- This setup enables a direct Ethernet link between the PYNQ-Z2 board and the laptop for development and communication.

Outcomes



Fig 7: Object detection from HDMI input source (Laptop)

Outcomes...contd

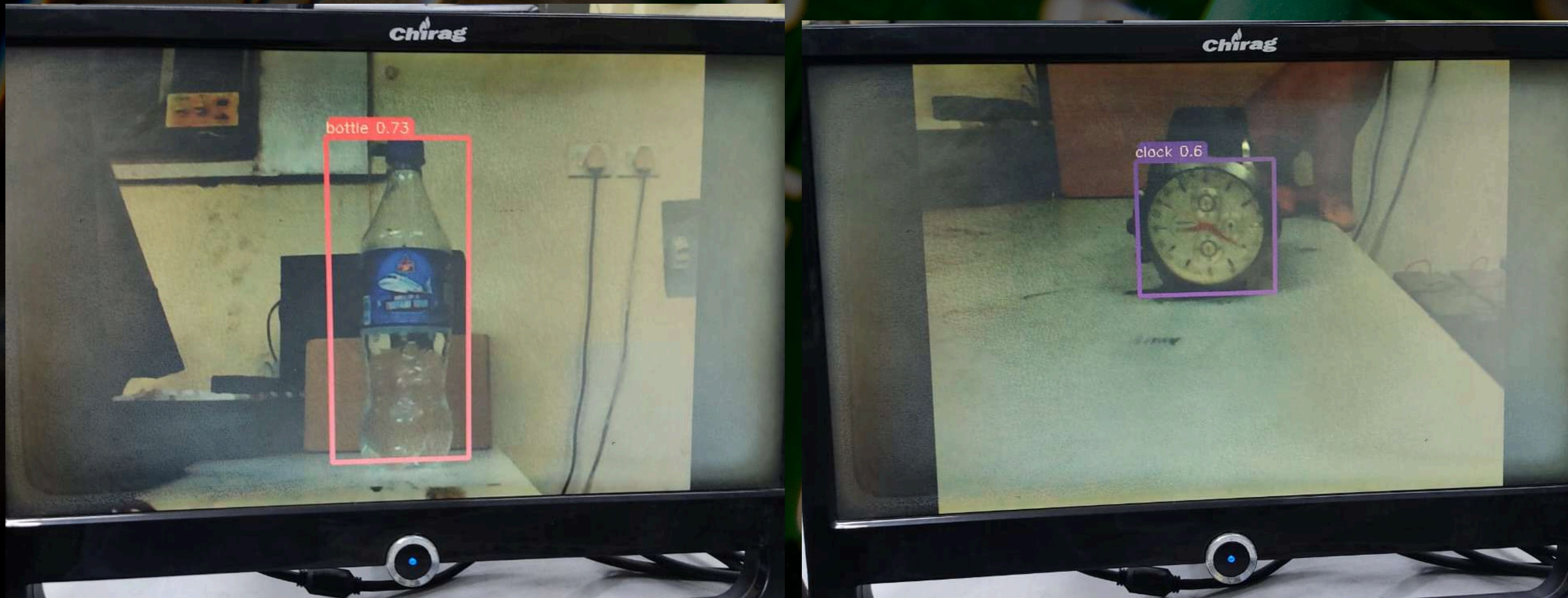


Fig 8: Object Detection Using Webcam USB

Outcomes...contd



Fig 9: Face Detection Using Webcam

FUTURE WORK

- The following project can be implemented in various aspects of real life, it can be upgraded in various field.
- The PYNQ Board takes time to process each frame and hence in this way a lot of frames is omitted until the next is picked up for processing.
- This can be improved by the use of pipelining. Through pipelining each and every frame will be processed and a continuous stream of video can be observed. This will also increase the processing speed.
- This will help us in achieving almost perfect and live image processing

CONCLUSION

- The hardware acceleration of object detection using the PYNQ Z2 board offers a promising avenue for optimizing computational performance in real-time image processing tasks.
- By leveraging the FPGA architecture of the PYNQ Z2 board, object detection algorithms can be offloaded from traditional CPU/GPU architectures, allowing for parallel processing and increased efficiency.
- This approach enables faster inference speeds, making it suitable for applications where low-latency processing is critical, such as autonomous vehicles, surveillance systems, and industrial automation.
- However, challenges such as FPGA programming complexity and resource constraints must be addressed to fully realize the potential benefits of hardware acceleration on the PYNQ Z2 board.

References

1. J. G. Pandey, A. Karmakar and S. Gurunarayanan, "Architectures and algorithms for image and video processing using FPGA-based platform," in 18th Int. Sym. on VLSI Design and Test VDATIEEE, pp. 1, 2014.
2. J. Rettkowski, A. Boutros and D. Göhringer, "HW/SW co-design of the HOG algorithm on a Xilinx Zynq SoC," Journal of Parallel and Distributed Computing, vol. 109, pp. 50–62, 2017.
3. R. Cumplido, M. Gokhale and M. Huebner, "Guest Editorial: Special issue on Reconfigurable Computing and FPGA technology," Journal of Parallel and Distributed Computing, vol. 133, pp. 359–361, 2019.
4. K. Vipin and S. A. Fahmy, "FPGA dynamic and partial reconfiguration: A survey of architectures, methods, and applications," ACM Computing Surveys, vol. 51, no. 4, pp. 1–39, 2019.
5. <https://www.kaggle.com/code/yasserh/object-detection-yolo-v4>
6. T. Saidani, "Deep Learning Approach: YOLOv5-based Custom Object Detection," Engineering, Technology & Applied Science Research, vol. 13, no. 6, pp. 12158–12163, Dec. 2023.

***THANK* YOU**